

Parallel Numerical Library

Debug report: what went wrong, and what each fault taught

Olajide Badejo

August 2026

Abstract

This is the engineering log of the project, kept from the first command and grouped here by theme. Each entry records the symptom as it presented, the root cause once found, the options considered, the fix and why it was chosen over the alternatives, and how it was verified.

It is a deliverable of equal rank to the main report because the main report contains only the conclusions, and the conclusions are not the part that is hard to reproduce. Several entries below describe faults that produced entirely plausible numbers, which is the failure mode worth documenting: an obvious crash teaches nothing, whereas a solver that converges in one iteration for the wrong reason will be believed.

It covers two bodies of work. The first is the build of release 1.0.0, where almost every entry is a thing that stopped the project compiling, converging or running. The second is the repair that produced release 1.1.0, which began with an audit of the released tree and is the larger half of this document. Its entries have a different character: hardly any of them was a failure. They are numbers that were wrong while looking right, provenance that could not be reproduced, claims smaller than their own measurement noise, and gates that were green because they checked nothing. A crash is found by running the program. A result that is wrong by a third, in the same direction as the figure it is divided by, is found only by counting.

Contents

1	Environment and toolchain	4
1.1	ENV-01: nvcc rejects the project compiler, and its own default too	4
1.2	ENV-02: a CMake setting that arrives too late	5
1.3	ENV-03: MPI version macros describe conformance, not capability	5
1.4	ENV-04: WSL2 hides the hybrid core topology entirely	5
1.5	ENV-05: the CUDA host compiler's libstdc++ hijacks the link	6
2	Numerics	7
2.1	NUM-01: the manufactured solution was an eigenvector	7
2.2	NUM-02: a default that was right for one solver and wrong for another	8
2.3	NUM-03: power iteration is the wrong way to choose a step	8
2.4	NUM-04: a tolerance below the arithmetic	8
2.5	MEAS-05: conjugate gradient reports the recurrence residual	9
2.6	NUM-05: the flag the bit identity claim rests on had nothing that could notice its absence	9
2.7	NUM-06: the relaxation factor a foreign stencil never received	10
2.8	NUM-07: a matrix allocates its square before deciding the order is legal	10
2.9	NUM-08: the Thomas solver writes past the end of an empty system	11
2.10	NUM-09: a bracket that underflows to zero is not a bracket	11
2.11	NUM-10: an integration that missed its tolerance by twelve orders and called itself converged	12
2.12	NUM-11: conjugate gradient calls an exact answer a proof that the operator is indefinite	12
2.13	NUM-12: Brent reports an exact root with the error estimate of the bracket it was found in	13
3	Concurrency	14
3.1	CONC-01: a destructor ordering hazard in the jthread pool	14
3.2	CUDA-02: the host was contracting to FMA and the device was not	14
3.3	CUDA-01: kernels in a header become duplicate device stubs	15
3.4	CUDA-03: a device to device copy given a host pointer	15

3.5	CONC-02: a counter incremented under the mutex, incremented without it, and read without it	15
3.6	CONC-03: a body that throws kills the process, or stops it forever	16
3.7	CONC-04: the shared topology is rewritten under the references it handed out	16
3.8	MEAS-12: a backend borrows the caller's thread and never gives it back	17
3.9	CUDA-04: a grid index that wraps, and a refused launch that reads like a fault	17
4	Distributed memory	18
4.1	MPI-01: the iterate is distributed but the result must not be	18
4.2	MPI-02: block ranges are not row ranges	18
4.3	MPI-03: a failure on one rank hangs the job instead of failing it	19
4.4	MEAS-09: the hybrid backend plotted its scaling curve against its rank count	19
5	Instrumentation and process	20
5.1	SWEEP-01: the resumable sweep was not resumable	20
5.2	SWEEP-02: two quadratic methods declared in a linear block	20
5.3	BUILD-01: CMake scans for modules that do not exist	21
5.4	STYLE-01: the specification itself contained an em dash	21
5.5	SWEEP-05: the resume check normalised a column the driver never normalised	21
5.6	SWEEP-06: a strict schema check with no repair, and eight columns queued up behind it	22
5.7	CLI-01: a mistyped flag terminates the driver, and a zero repetition count crashes it	22
5.8	CLI-02: two ways out of the parser that the caller could not catch	23
5.9	TEST-01: the test framework had three defects, and nothing tested the test framework	23
6	Provenance and measurement validity	24
6.1	PROV-01: every published result row is stamped dirty	24
6.2	PROV-02: an excluded directory that no later negation can re include	25
6.3	MEAS-01: Jacobi copies the whole state on the calling thread, every iteration	25
6.4	MEAS-02: Richardson evaluates the residual twice per iteration	26
6.5	MEAS-03: the symmetric methods were charged one sweep, and the mitigation the code claimed did not exist	26
6.6	MEAS-04: a relaxation factor recorded on rows that never used one	27
6.7	MEAS-06: the byte model is undercounted, and the triad it is divided by pays the same charge it does not count	27
6.8	MEAS-07: the probe's run to run spread is wider than the band it selects on	28
6.9	MEAS-08: two pinning policies bound nothing and said they had	28
6.10	MEAS-10: the timed region allocated on every call, and the state was only the largest part of it	29
6.11	MEAS-11: dispersion was collected on every row and read by nothing	29

6.12	PROV-03: two generations in one summary, told apart by the order they were appended in	30
6.13	PROV-04: the committed manifest described a different session	30
6.14	PROV-05: the manifest named a compiler that built nothing	31
7	Build, install and continuous integration	32
7.1	BUILD-02: the one flag target carried the machine's architecture into every consumer's build	32
7.2	BUILD-03: the export set refuses a target that links a static library it does not contain	32
7.3	BUILD-04: a private dependency of a static library is not private to the export set	33
7.4	BUILD-05: the self check for the allocation counter was deleted by the compiler it was meant to check	33
7.5	BUILD-06: a unit test named a backend the build is allowed not to contain . .	34
7.6	BUILD-07: a default argument on a virtual function, and the override that omitted it	34
7.7	CI-01: the CUDA job reported success having compiled zero CUDA	35
7.8	CI-02: the reports job never compared the reports it rebuilt against the ones it ships	35
7.9	CI-03: the compiler, the runner image and every action were whatever the day supplied	35
8	The documents themselves	37
8.1	DOC-01: a hand typed table under the sentence saying nothing was typed by hand	37
8.2	DOC-02: no prior art and no baseline	38
9	What the faults have in common	39
9.1	What the repair added to that list	39

Chapter 1

Environment and toolchain

Four of these five changed the structure of the build. They are first because they were first: the project could not compile until they were resolved.

1.1 ENV-01: nvcc rejects the project compiler, and its own default too

Symptom. Configuration failed inside `project(... CUDA)` with a compiler identification error. Invoking `nvcc` by hand on a trivial kernel with the system default host compiler failed differently, inside a system header:

```
/usr/include/x86_64-linux-gnu/c++/15/bits/c++config.h(586):  
    error: expected a "("  
        if consteval { return true; } else { return false; }
```

Root cause. Two separate problems presenting as one. First, CUDA 13.3 supports GCC only up to 15, and this project uses GCC 16 for the host code. Second, even at GCC 15, which `nvcc` claims to support, `libstdc++ 15` uses the C++23 `if consteval` statement in `c++config.h`, and the EDG frontend inside `nvcc` cannot parse it. The nominally supported combination is also broken.

Options. Dropping the host code to GCC 14 throughout was rejected: it costs `<mdspan>` and drops the OpenMP level from 5.2 to 4.5, both of which the project uses. Passing `-allow-unsupported-compiler` was rejected because the failure is a genuine parse error in a system header, not a version check being cautious.

Fix. Compile only the `.cu` files with `nvcc -ccbin g++-14 -std=c++20`, and declare every CUDA entry point `extern "C"` taking plain pointers and scalars. No `libstdc++` type crosses the boundary, so the two compilers never have to agree on a C++ ABI, only on the platform C ABI, which they do by definition. This also enforces at the ABI level the rule that backend files never leak their model's types.

Verification. A kernel built this way links against GCC 16 host code using `<mdspan>` and runs on the GPU returning the expected values.

1.2 ENV-02: a CMake setting that arrives too late

Symptom. After ENV-01, configuration still failed identically even though `set(CMAKE_CUDA_HOST_COMPILER g++-14)` was present in `CMakeLists.txt`.

Root cause. The line sat after `project(... LANGUAGES CXX CUDA)`. Compiler identification runs inside `project()`, so the setting arrived too late and identification used the default host compiler, which `nvcc` rejects.

Fix. Set it as a cache variable before `project()`, with a comment saying why so it cannot be tidied back down.

1.3 ENV-03: MPI version macros describe conformance, not capability

Symptom. The specification names MPI 5.0 as its reference and asks which of its features the installed library implements. `MPI_Get_version` returns 3.1 and the `MPI_VERSION` compile time macro is 3.

Root cause. OpenMPI 5.0.x does not claim conformance to MPI 4.0 as a whole, so it leaves the version macro at 3.1, but it ships many MPI 4.0 entry points. Link probes confirmed that `MPI_Session_init`, `MPI_Comm_create_from_group`, `MPI_Isendrecv`, `MPI_Allreduce_init`, `MPI_Info_create_env`, `MPI_Barrier_init` and `MPI_Comm_idup_with_info` all resolve.

Fix. Probe for symbols with `check_cxx_symbol_exists` rather than testing the version macro. Guarding on the macro would disable functionality that demonstrably works; assuming the functions are present would break on a stricter library. The core path uses only MPI 3.1 guaranteed API.

1.4 ENV-04: WSL2 hides the hybrid core topology entirely

Symptom. Objective 3 asks for the performance versus efficiency core knee on an i7-14700K, which has eight performance cores of two threads each and twelve efficiency cores. Inside the guest, `lscpu` reports fourteen uniform cores, every logical processor reports the same 2048K L2 and `cpu_capacity` of 1024, the hybrid flag is absent from `/proc/cpuinfo`, and `/sys/.../cpufreq` does not exist.

Root cause. WSL2 is a Hyper-V virtual machine and synthesises a homogeneous topology. Separately, `sched_setaffinity` binds a thread to a guest virtual processor and the hypervisor remains free to place that processor on any host core, so pinning is a hint about the guest and not a guarantee about silicon.

Options. Assuming the conventional enumeration, that host processors 0 to 15 are performance core threads and 16 to 27 are efficiency cores, was rejected: it is unverifiable from inside the guest and would produce confident labels with nothing behind them. Booting to bare metal was out of scope, the specification fixing WSL2 as the environment.

Fix. Measure rather than ask, and take the knee from a measurement that survives virtualisation. The topology probe times an identical kernel on each logical processor and reports a two group split only when the throughputs separate by a margin dominating the within group spread; otherwise it says so and the core type policies decline to bind. The knee is read from the aggregate scaling curve, which depends only on how many cores are engaged and not on which.

1.5 ENV-05: the CUDA host compiler's libstdc++ hijacks the link

Symptom. Enabling the CUDA backend broke a link that had worked throughout the project, with undefined references to `std::__detail::__wait_impl`, `std::__detail::__notify_impl` and `_M_setup_proxy_wait`, all reached from the `jthread` pool. Nothing in the CUDA code uses threads, and the failing objects were compiled by GCC 16.

Root cause. Those symbols are the out of line half of GCC 16's atomic wait and notify support, which `std::jthread` and `std::barrier` need. CMake detects the CUDA implicit link directories by asking `nvcc`, and `nvcc` drives `g++-14`, so the list contained `/usr/lib/gcc/x86_64-linux-gnu/14`. That `-L` landed ahead of the search path `g++-16` adds for itself, so `-lstdc++` resolved to GCC 14's copy.

Two wrong turns before finding it. The first guess was that spaces in the project path were to blame, which a controlled test disproved. The second was that the link was driven by the wrong compiler, which setting the linker language did not fix because `g++-16` was already driving it. The problem was the search order, not the driver.

Fix. Filter every version specific GCC directory out of the CUDA implicit link directories, which is safe precisely because the link is driven by the C++ compiler and it contributes its own. The configure step prints each directory it drops, so the filtering is visible.

Chapter 2

Numerics

These are the entries worth reading twice. Each produced a plausible number rather than an obvious failure.

2.1 NUM-01: the manufactured solution was an eigenvector

Symptom. The test asserting conjugate gradient needs $O(n)$ iterations on the model problem failed: the count did not grow at all between a 31 and a 63 grid. Measured directly, the method converged in exactly one iteration at every size.

Root cause. Not a bug in the method. The eigenvectors of the five point stencil are $\sin(p\pi x)\sin(q\pi y)$, and the manufactured solution $u = \sin(\pi x)\sin(\pi y)$ is exactly the lowest of them. With a zero initial guess the residual is a single eigenvector, so the Krylov subspace is one dimensional and the first step is exact. The problem was degenerate, not the method fast.

The same choice cuts the other way for the stationary methods: the slowest decaying mode of the Jacobi iteration matrix is that same lowest mode, so a single mode source isolates the asymptotic rate cleanly and reproduces the closed form spectral radius to six digits. One choice is ideal for one family and useless for the other.

Options. Changing to a polynomial such as $x(1-x)y(1-y)$ was rejected: its fourth derivatives vanish, so the five point stencil becomes exact for it and the second order accuracy test loses its subject. A sum of a few sine modes was rejected because it only moves the problem, a sum of three eigenvectors making the method terminate in three steps.

Fix. Provide both right hand sides and use each where it is honest. The manufactured sine source keeps the closed form solution and is used by the discretisation error and stationary rate tests; a seeded spectrally rich source excites the whole spectrum and is used for anything involving conjugate gradient and for the benchmark sweep, where a degenerate spectrum would flatter one method over the others.

Verification. With the rich source the count grows from 41 to 93 as the grid doubles, consistent with the $O(n)$ bound. A dedicated test now asserts the one iteration behaviour on the single mode source, so the trap stays recorded rather than merely avoided.

2.2 NUM-02: a default that was right for one solver and wrong for another

Symptom. Richardson reported divergence on both a 4 by 4 dense system and a 15 by 15 Poisson problem, while its own dedicated convergence test passed.

Root cause. The relaxation factor defaulted to one. The tests that passed set it explicitly to zero, meaning "ask the solver"; the tests that failed left the default. Richardson with $\omega = 1$ on the Poisson operator has iteration matrix $I - A$, whose eigenvalues reach $1 - 8 = -7$, so the spectral radius is seven and it diverges immediately. The default had been chosen thinking of SOR, where one means Gauss Seidel and is harmless.

Fix. The default is now zero, meaning each solver picks its own: Young's closed form optimum for SOR, one for SSOR, and the reciprocal of the Gershgorin bound for Richardson. Zero is the only default correct for every member of the family. The general lesson is that a shared options struct wants defaults meaning "ask the component", not defaults that suit whichever component was written first.

2.3 NUM-03: power iteration is the wrong way to choose a step

Symptom. The first Richardson implementation estimated the largest eigenvalue by power iteration and diverged even when asked to choose its own step.

Root cause. Two faults. The Rayleigh quotient approaches $\lambda_{\max}$ from below, so an unconverged estimate yields a step larger than $1/\lambda_{\max}$ and can leave the convergence interval entirely. Separately, the starting vector was written across the whole padded grid including the Dirichlet boundary ring, which corrupts the operator.

Fix. Replaced by a Gershgorin bound, which overestimates the spectral radius by construction, so its reciprocal is always strictly inside the convergence interval. It is exact for the stencil, free to compute, deterministic, and identical on every backend, none of which the power iteration was.

2.4 NUM-04: a tolerance below the arithmetic

Symptom. The discretisation error tests asked for a relative residual of 10^{-13} and stopped converging when the solver they used changed from Gauss Seidel to optimally relaxed SOR.

Root cause. Not a regression. Stationary iterations have a rounding limited residual floor, and over relaxation raises it because it amplifies rounding noise. Measured on this operator in double precision:

grid	forward Gauss Seidel	SOR at optimal omega	conjugate gradient
15 by 15	4.4e-15	9.5e-15	8.1e-17
31 by 31	1.7e-14	5.6e-14	8.6e-17
63 by 63	7.0e-14	2.9e-13	9.4e-17

The requested 10^{-13} sat below the floor at 63 by 63, so the iteration could never report convergence however long it ran.

Fix. Those tests now ask for 10^{-11} , three orders above the measured floor and seven below the discretisation error of about 2×10^{-4} they actually measure. The table above is quoted in the test comment so the number is not mistaken for a guess.

2.5 MEAS-05: conjugate gradient reports the recurrence residual

Symptom. Found while writing the check that every method reports the residual at the iterate it returns. Eleven of the twelve methods pass it exactly. Conjugate gradient does not: after ten fixed iterations at 63 squared it reports 7.2×10^{-15} while $b - Ax$ at the solution it returned is 1.1×10^{-13} .

Root cause. Not a defect. The method updates its residual by the recurrence $r_{k+1} = r_k - \alpha_k A p_k$ rather than recomputing $b - Ax_{k+1}$, which is the standard formulation and the whole reason it costs one operator application per iteration instead of two. The two agree in exact arithmetic and drift apart as rounding accumulates, and the drift here is at the scale rounding predicts.

Options. Recomputing the true residual would double the cost of the method and make its evaluation count $2k + 1$, which is a worse comparison rather than a better one. Recomputing it once at the end, for the report only, would make the reported residual disagree with the convergence test the run actually stopped on.

Fix. None. The fact is recorded, and the unit case that asserts the property for the other eleven exempts conjugate gradient with the reason written at the exemption, so a future change that made it recompute would have to clear the exemption deliberately.

2.6 NUM-05: the flag the bit identity claim rests on had nothing that could notice its absence

Symptom. No failure. Found by reading the contract rather than by watching something break. The library promises identical numerics across every execution model to the last bit, and that promise depends on `--ffp-contract=off`. Removing the flag and rebuilding left every test green, because the equivalence suite compares backends against each other inside one process, all compiled the same way. They would all contract, they would contract identically, and the suite would pass while the claim had quietly become false against the device path, which is compiled `--fmad=false` and does not contract.

Root cause. A flag asserted nowhere is a comment. The suite's comparison is internal, so it is invariant under any change that affects every backend equally, and contraction is exactly such a change.

Fix. A header level probe that evaluates an expression whose value differs by one unit in the last place under contraction and refuses to compile when the answer is the fused one, plus a device side pair of cases that compare the library's probe against the same source built with fusion enabled and assert the two are different doubles. That last assertion is what makes the comparison a comparison rather than a tolerance in disguise.

Verification. Three cases, on the target machine's device. The library probe returns exactly 2^{-26} ; the fused build of the same source returns exactly $2^{-26} + 2^{-54}$; the third case asserts those are different doubles. It runs with or without a device present, because it is about the numbers.

2.7 NUM-06: the relaxation factor a foreign stencil never received

Symptom. No failure, and this one could not have produced one: the library shipped no problem it was wrong for. `Sor::resolve_relaxation` answered "what relaxation factor should SOR use when the caller asked for none" with a downcast to `Poisson2D`, falling back to one for anything else.

Root cause. The knowledge lives in the wrong class. Which factor is right is a property of the operator's spectrum, so the operator is the only object that can answer, and the solver asked by testing the operator's type against a list of one. A downcast in place of a virtual is a closed world assumption written down: correct exactly while the set of problems is the set the author enumerated, and failing silently rather than loudly on the day it is not, because the fallback is a legal value.

Fix. `Problem::suggested_relaxation()` returns one and says why: one makes SOR into Gauss Seidel, which converges on any symmetric positive definite or strictly diagonally dominant system and claims nothing about a spectrum the base class does not know. `Poisson2D` overrides it and returns the same member of the same object the downcast used to reach, so the number is bit identical and not merely equal.

Verification. Every SOR iterate in the sweep is unchanged, which is the property that matters: a factor that was only approximately the old one would have moved every one of them.

2.8 NUM-07: a matrix allocates its square before deciding the order is legal

Symptom. An order of minus one thousand allocated the eight megabytes of a thousand by thousand matrix and then reported that the order was negative. Under the sanitizers, an order above $2^{31.5}$ reported signed integer overflow in the member initialiser.

Root cause. The precondition was in the constructor body and the storage was built in the member initialiser list, which runs first. A negative order squares to a positive number, so the allocation succeeded before the check could refuse it. The rectangular case in the QR matrix was worse: a negative product cast to an unsigned size is about 1.8×10^{19} elements, so it threw a length error from inside the vector rather than the documented argument error.

The overflow is the more serious half, because signed overflow is undefined behaviour rather than wraparound: at order 2^{32} the wrapped product is exactly zero, so the ordinary build allocated nothing, constructed a matrix claiming an order of four billion over an empty array, and reported no error at all.

Fix. A `checked_extent` helper that refuses a negative extent and tests the product against the index limit by division, so finding out that the product would overflow does not overflow on the way. Both matrix types build their storage from it and their constructor bodies are now empty.

Verification. Two cases in the allocation counting suite, which gained the size of each request alongside the count: "allocated nothing" is not the assertion here, since the refusal builds the sentence it carries, so what separates the two states is eight megabytes against a hundred bytes. Both pass in the ordinary build and under the sanitizers, where the overflow report is gone.

2.9 NUM-08: the Thomas solver writes past the end of an empty system

Symptom. Four empty spans, which the length check accepts because they are all the same length, reported a heap buffer overflow under the address sanitizer.

Root cause. The forward sweep begins outside the loop, because the first row of the tridiagonal recurrence has no sub diagonal term. With no rows at all, that first step still ran.

Fix. An early return for the empty system, after the precondition checks and before the sweep, with the reason recorded beside it.

Verification. Two cases: the empty system, and the system of one unknown beside it, because the second is the case a careless fix to the first would break.

2.10 NUM-09: a bracket that underflows to zero is not a bracket

Symptom. A function positive everywhere, handed to a bracketing root finder over an interval containing no root, was accepted as bracketing one.

Root cause. The precondition read `fa * fb <= 0`. The intent is "the ordinates have opposite signs, or one of them is a root", and the product is a proxy for it that fails in the small: two ordinates near 10^{-154} have a product that rounds to positive zero, and zero satisfies the test. That is well inside the range a scaled residual reaches near a root, so the failure is not exotic.

Fix. Explicit sign predicates at all three sites. Zero on either endpoint is still a bracket, because it is already a root and the finders return it as one, and a NaN is decided explicitly rather than left to whichever sign bit the platform's NaN carries, which preserves the behaviour the product spelling had.

Verification. The underflowing pair is refused by both bisection and Brent; a genuine sign change whose product overflows to negative infinity is still accepted, which is the regression the fix could plausibly have introduced; and an endpoint that is exactly zero, in both spellings of zero, is still accepted and returned as the root.

2.11 NUM-10: an integration that missed its tolerance by twelve orders and called itself converged

Symptom. Decay at rate fifty over the unit interval, a tolerance of 10^{-12} and a minimum step far too coarse for it. Ten steps, every one out of tolerance, and the result reported itself converged. The diagnostics interface that exists precisely so a caller cannot proceed on a bad answer passed it.

Root cause. The acceptance test accepts a step when the error is small enough *or* the step has hit its floor, and the second half is right: once the controller cannot shrink the step further, refusing it means an integration that cannot advance at all. The fault is that nothing recorded which half had accepted. Convergence was set by reaching the endpoint alone, so a run that got there over steps the controller had given up on was indistinguishable from one that met its tolerance everywhere.

Fix. A distinct stop reason for the step floor, set when a step whose error exceeds one is accepted because the step size has bottomed out. Reaching the endpoint is still necessary and is no longer sufficient.

Verification. A case that forces the floor rather than approaching it, and which first asserts that the endpoint really was reached and the worst error really was above one, so that a future change which stopped exercising the floor fails rather than passing vacuously.

2.12 NUM-11: conjugate gradient calls an exact answer a proof that the operator is indefinite

Symptom. The one unknown boundary case added to the equivalence suite failed on the first backend it reached, with a numerical failure saying that a search direction of zero curvature proves the operator is not positive definite. The operator is the one by one matrix [4].

Root cause. The breakdown check tests the curvature against zero, and a curvature of zero has two causes the check does not separate. One is a genuinely indefinite operator, which is what the message describes. The other is a search direction that is the zero vector, which has zero curvature under every operator there is, including the most positive definite one imaginable. The recurrence produces the zero direction as soon as it has solved the system exactly, which at one unknown is after one iteration.

Fix. A guard at the top of the loop: an exactly zero residual is convergence and is returned as such, before the curvature is ever formed.

Verification. The boundary case that found it, at one unknown on both problems, every solver in the registry, every shared memory backend and every worker count this machine has.

2.13 NUM-12: Brent reports an exact root with the error estimate of the bracket it was found in

Symptom. Found by a fuzz target on draw 23 of its fixed seed: Brent reported convergence with a bracket half width of about 7.6×10^{199} against a tolerance of 10^{-12} .

Root cause. The loop ends on either of two conditions, a small enough bracket or an ordinate that is exactly zero, and the error estimate was written before the test from the bracket alone. When the second condition ends the loop the answer is exact and the estimate describes the interval it happened to be found in.

Fix. One line: an exact root reports an error estimate of zero, which is what it has.

Verification. Twenty thousand inputs per target from a fixed seed. Worth recording separately: the first thing that fuzzer found was a defect in the test rather than in the code. An earlier version of the harness asserted that a converged run must have a small residual, and was refuted in forty draws, because the tolerance is an absolute bound on the bracket half width and not on the residual. The harness was wrong, it now says so in a comment, and that is the more common outcome of pointing a fuzzer at a library.

Chapter 3

Concurrency

3.1 CONC-01: a destructor ordering hazard in the jthread pool

Symptom. Found by inspection during review of the shutdown path, before it could reproduce.

Root cause. Workers wait on a barrier and, once released, read an atomic `stopping_` flag before returning. `std::jthread` joins in its own destructor, and members are destroyed in reverse declaration order, which placed `stopping_` after the thread vector. A worker still reading the flag while it was being destroyed would be a use after free, in a window that would almost never reproduce and would be attributed to something else when it did.

Fix. Join every worker explicitly in the destructor body after releasing the barrier, before any member can be destroyed. Relying on the `jthread` destructor was tidier to read and wrong.

3.2 CUDA-02: the host was contracting to FMA and the device was not

Symptom. GPU Jacobi and GPU red black Gauss Seidel came out bit identical to the CPU, but GPU red black SOR differed by 1.6×10^{-18} , one bit in the last place.

Root cause. The device is compiled with `--fmad=false`, so it evaluates a multiply and an add separately. The host had no such restriction, and the SOR update has the shape $ab + cd$, which GCC is free to contract into a fused multiply add. It did. Jacobi has no such shape, which is why only SOR disagreed and why the discrepancy looked mysterious rather than systematic.

Options. Comparing SOR to a tolerance was rejected: the central claim of the library is that identical numerics run over every execution model, and "identical unless the compiler decided to contract" is a much weaker claim that happens to be invisible in the test output. Enabling FMA on the device was rejected because it would fix SOR and break Jacobi.

Fix. Turn contraction off on both sides, `-ffp-contract=off` on the host alongside `--fmad=false` on the device. The cost is nil in practice, these kernels being bandwidth bound, and the gain is that results no longer depend on whether a particular compiler on a particular target felt like contracting.

Verification. All three GPU sweeps are now bit identical to the CPU, and the device and host red black runs agree on iteration count exactly.

3.3 CUDA-01: kernels in a header become duplicate device stubs

Symptom. Multiple definition of `__device_stub__ZN8pnl_cuda11xpy_kerneLEPKddPdii`, reported against a generated file with a name like `tmpxft_00280375_00000000-6_stream_probe.cudafe1.cpp`.

Root cause. The shared kernels were defined in a header. A `__global__` function in a header is emitted into every translation unit that includes it, along with its host side stub, so two `.cu` files including it produced two definitions of each. The error names a mangled stub in a generated file and gives no hint that a header is responsible.

Fix. The shared header now holds error handling, launch geometry and declarations only. Each kernel is defined in exactly one `.cu`, and anything needed across files goes through a plain launcher function.

3.4 CUDA-03: a device to device copy given a host pointer

Symptom. Every device solve failed immediately with `cudaMemcpy(...) failed: invalid argument`.

Root cause. A plain slip: the source should have been the device copy made two lines earlier, not the caller's host array. Recorded because the class of mistake is common at a C ABI boundary, where pointers carry no indication of which address space they belong to.

3.5 CONC-02: a counter incremented under the mutex, incremented without it, and read without it

Symptom. The pthreads pool's pinning failure counter was a plain integer with three accesses and no single rule: the constructor incremented it with no lock, each worker incremented it under the mutex, and the accessor read it with no lock at all.

Root cause. The constructor's increment runs before any thread is created and races with nothing. The workers' increments are under the mutex and race with nothing either. The only pair that is a data race by the letter of the standard is the guarded increment against the unguarded read, and until the phase that needed the counter nothing called the accessor, so the read never happened and the race had no second access to be a race with.

Fix. The counter is atomic, incremented and read with relaxed ordering. Relaxed is enough because it is only added to, never used to publish anything, and only read after a happens before edge the pinning handshake already supplies. The mutex is not taken for it anywhere.

3.6 CONC-03: a body that throws kills the process, or stops it forever

Symptom. A dispatch over four workers whose body throws from exactly one chunk, with the exception caught around the dispatch. Six runs, one per pool and per throwing chunk: some terminated the process, and some hung.

Root cause. Three boundaries an exception may not cross, and the library crossed all three. A thread function that lets an exception escape terminates the process. An exception that escapes a chunk before the worker reaches the barrier leaves every other worker waiting on a participant that will never arrive. And an exception thrown inside an OpenMP structured block must not leave it.

Fix. An exception relay: a noexcept wrapper around each chunk, where the first thrower wins an atomic exchange and publishes an exception pointer with a release store, and the dispatching thread reads it with an acquire load and rethrows, clearing the relay so the backend is usable again. The four dispatching paths that cross a thread boundary use it; the serial and distributed backends run their chunks on the calling thread, where a throw already propagates, and are unchanged.

Verification. Every pool the build has, at one, two and eight workers, throwing from the first chunk and from the last, asserting the message, that every chunk which did not throw ran, and that a further dispatch afterwards still covers the whole grid. Its test entry carries a thirty second timeout rather than the suite's fifteen minutes, so the deadlock cannot come back as a hang.

3.7 CONC-04: the shared topology is rewritten under the references it handed out

Symptom. Take the cheap topology report, read it, ask for the probed one, then read the first again: it had changed underneath the reference.

Root cause. One report object at namespace scope served both passes. The cheap pass filled it under one once flag and the probing pass replaced it wholesale under a second, so a reference handed out by the first call described something else after the second.

Fix. Two caches, each filled inside its own call once and never touched again, with the accessor returning a reference to whichever the caller asked for. Call once is the happens before edge, so a reader cannot see a partly written report, and a reference handed out earlier keeps describing what it described.

Verification. A case that holds the cheap reference across a probing call and requires its address, its processor count, its core leader list and its verdict to be unchanged, then requires both caches to answer with the same objects on a second call.

3.8 MEAS-12: a backend borrows the caller’s thread and never gives it back

Symptom. A pinning case passed for the wrong reason. Every shared memory pool makes the calling thread worker zero and binds it, which is correct and is the point of the pinning repair. What none of them did was put the calling thread’s affinity mask back on destruction, so a test process that built and destroyed several pools ended up pinned to one processor, and the later case that asserts a policy is refused was reading a topology verdict taken through a narrowed mask.

Fix. An affinity guard object that captures the calling thread’s mask on construction and restores it on destruction, with a restore that is const, idempotent and noexcept so it can be called from several threads and from a destructor.

Verification. A case that builds and destroys each pool under each policy at two and four workers and requires the visible processor set afterwards to equal what it was before, asserting on the way through that the backend really did bind while it existed, so a fix that restored the mask too early would fail here too. It runs first in its file, because the case downstream is what was reading the leak.

3.9 CUDA-04: a grid index that wraps, and a refused launch that reads like a fault

Symptom. Two faults on the device path, neither of which any existing test could reach: an index expression that overflows a signed 32 bit integer at large grids, and a launch geometry the driver refuses being reported as an internal error.

Root cause. The index arithmetic is 32 bit throughout the device sources, which is the right choice for it, and nothing anywhere said what that implies about the largest problem the kernels can address. An unstated bound is one nobody can check.

Fix. A named maximum side length, with the derivation beside it: the largest index the kernels form is $n(n + 2) + n$, and 46338 is the largest n for which that stays inside a signed 32 bit integer. The entry point refuses anything above it before it allocates or launches, and the driver refuses it before it even looks for a device, since it is a property of the request and not of the machine. The interior count is formed as a 64 bit product and narrowed afterwards, so the multiplication cannot overflow whatever it is handed.

Verification. Two cases on the target machine’s device, through a test hook that launches a helper with the block geometry it is given. A geometry four times the device limit is refused with the wording the case requires, and a legal geometry through the same path is accepted, so the probe is reporting the geometry rather than always failing. The bit identity cases either side of it are unchanged, which is the evidence that no measured number moved.

Chapter 4

Distributed memory

4.1 MPI-01: the iterate is distributed but the result must not be

Symptom. At two and four ranks, four separate test cases failed at once: order free solvers differed from serial by 3×10^{-4} , ordered solvers were not bit identical, dense systems differed by 0.998, and the discretisation error came out as 1 instead of 2×10^{-4} .

Root cause. One cause behind all four. During the iteration a rank needs only its own rows plus a halo, and the code was correct in that respect: residual norms and convergence detection were right on every rank. But the returned solution vector was left distributed, each rank holding only its own rows, and the tests compared the whole vector.

Options. Gathering after every sweep was rejected: it would dominate the communication measurement and would measure the wrong thing entirely, since the iteration genuinely does not need it.

Fix. A `synchronise` method on the problem, called once at the end of each solve. For the padded grid the owned rows form a single contiguous run of the array, so the gather needs no knowledge of the padding.

Verification. All MPI tests pass at one, two and four ranks, including bit identity against serial for the pipelined ordered sweeps.

4.2 MPI-02: block ranges are not row ranges

Symptom. Caught during review of the dense path rather than by a failing test.

Root cause. A rank's share of the *blocks* in a block method covers a different set of rows than its share of the *rows* in a point method, whenever the block count differs from the unknown count. A gather that assumed otherwise would have collected the wrong segments and corrupted the vector silently.

Fix. A general `gather_rows` primitive that collects each rank's range from the ranks themselves rather than assuming it, with the block derived range passed explicitly at the one call site where the two differ.

4.3 MPI-03: a failure on one rank hangs the job instead of failing it

Symptom. With a fault injected on rank one, from a point that case had collectives after, a two rank run did not fail. It hung until the test timeout.

Root cause. Two places, one shape. An error on one rank is caught locally, and the ranks that did not see it are left in a collective that has lost a participant. The job then makes no progress and reports nothing, which is worse than a failure, because a timeout says only that something took too long.

Fix. The driver aborts the whole job from the catch when the backend is distributed, after flushing, so one rank's error takes the job down with a status a caller can read.

Verification. The same injection now ends the job with that status, and a distributed run whose configuration is rejected fails rather than hanging.

4.4 MEAS-09: the hybrid backend plotted its scaling curve against its rank count

Symptom. Every hybrid row in the published summary had a worker count equal to its rank count. All thirty read five workers, five ranks and four threads per rank, so a configuration running twenty threads across the job was recorded at five workers and compared, on the same axis, against rows recorded at twenty.

Root cause. The hybrid constructor sets the configured worker count to ranks times threads and says in a comment that the product is what the scaling curve is plotted against. It then did not override the accessor, which is inherited from the distributed backend and returns the rank count, and the result row is written from the accessor rather than from the configuration. The right number was computed, stored in the field the row does not read, and documented in a comment describing a behaviour the class did not have.

Fix. The hybrid backend overrides the accessor and returns the product. The sweep driver's prediction of that row's identity clamps the thread count before multiplying, so a resumed sweep matches. The comment now describes what the code does.

Verification. Two ranks at four threads per rank writes eight workers; the sweep's own shape, five ranks at four threads, writes twenty.

Chapter 5

Instrumentation and process

5.1 SWEEP-01: the resumable sweep was not resumable

Symptom. Restarting the sweep reported "38 already complete" and then immediately began re-running the first configuration, a four minute solve that was already recorded.

Root cause. The skip test was in the right loop but on the wrong side of the work. Rows were parsed from the binary's output and only then compared against the completed set, so every configuration was executed in full and the only thing skipped was writing the row. The sweep was resumable in the sense that it did not corrupt the summary, and in no other sense. The reason it was written that way is that the identity tuple was taken from the emitted row, which does not exist until the run has happened. The circularity was invisible because the counters still reported plausible numbers.

Fix. Predict the identity from the declared configuration before anything is launched. The predictions that are not simply the declared value are the places this could go wrong again, so they are named in the code: the serial and CUDA backends report one worker whatever was requested, and the hybrid backend reports ranks times threads. A wrong prediction costs one redundant run, after which the real row makes the match exact, so the failure mode is waste rather than a missing measurement.

The commit is now probed from the binary rather than read from the first existing row, since a stale commit in the resume check would silently skip configurations whose code had changed.

Verification. Re-running a completed block reports 36 skipped in under a second, against roughly ten minutes of recomputation before.

5.2 SWEEP-02: two quadratic methods declared in a linear block

Symptom. Caught by arithmetic rather than by waiting. A sweep block intended for the methods whose iteration count grows like n listed symmetric Gauss Seidel and block Gauss Seidel at grid sizes 511 and 1023.

Root cause. Neither is $O(n)$. Symmetric Gauss Seidel at a relaxation factor of one, and line Gauss Seidel, are both $1 - O(h^2)$ and need of order n^2 iterations. At 1023 that is about a million sweeps over a million unknowns, hours per configuration, for a number the closed form already predicts.

Fix. The block now lists only optimally relaxed SOR, its red black form and conjugate gradient, with a comment giving the reason so the list is not helpfully extended later. A per configuration timeout was added to the sweep driver, so a future misdeclaration costs fifteen minutes rather than a night.

5.3 BUILD-01: CMake scans for modules that do not exist

Symptom. Every compile line carried `-fmodules-ts` and `-fdeps-format=p1689r5`, and the first build failed inside that machinery.

Root cause. From C++20 onward CMake scans each translation unit for module dependencies by default. This project is entirely header based and declares no modules, so the scan is pure cost and routes GCC through its experimental modules path.

Fix. `set(CMAKE_CXX_SCAN_FOR_MODULES OFF)`.

5.4 STYLE-01: the specification itself contained an em dash

Symptom. The dash checker's first run over the tree reported one violation, in the build specification, in the sentence asking for the personal report to be thorough.

Root cause. The document setting out the no dash ground rule contains one em dash in its own prose.

Fix. The specification is preserved for provenance with that one character replaced by a comma, and the substitution is recorded here rather than made silently, because the input to the build is part of the record.

5.5 SWEEP-05: the resume check normalised a column the driver never normalised

Symptom. Nothing visible, which is why it survived a release. The sweep is resumable, the published summary holds twenty four device rows, and every sweep re ran all twenty four of them and wrote the same numbers back.

Root cause. One line. The predicted identity of a device row normalised the backend name to `cuda` while the driver wrote the name it was given, so the predicted tuple never matched the recorded one and the skip test never fired.

Fix. The prediction carries the backend name unchanged and normalises the fields the driver actually normalises. Two further such fields were being added in the same commit, and adding a second normalised field beside a first one that is wrong is how a defect becomes a convention, which is why the repair landed there rather than in the phase that first needed them.

Verification. The same dry run against the same summary reports the twenty four rows as already present, where before it reported them as work to do.

5.6 SWEEP-06: a strict schema check with no repair, and eight columns queued up behind it

Symptom. The driver compares the header of the summary it is merging into against the header the binary emits, with strict list equality, and on any difference tells the operator to move the file aside and exits. The repair release adds eight columns to that header. Every one of them, on the day it lands, breaks the sweep against the committed summary and therefore breaks the full build, until a re measurement finishes. Stopping anywhere in between leaves the repository strictly worse than the release it started from: it does not build a report.

Root cause. The check is right and it is half a mechanism. Mixing two schemas in one file shifts every field after the first difference and produces a file that reads without error and means something else, so refusing is correct. There is no backfill anywhere, though, so the only remedy the message offers is to throw the measurements away, and a guard with no matching repair turns every schema change into a re measurement.

Fix. A migration script that adds the columns the binary has gained, from a table of declared defaults, refuses on a column that was removed rather than guessing what to do with it, and writes through the same atomic temporary file and rename the driver uses. A flag calls it instead of exiting, and a dry run does the header check and the resume calculation and stops, so the state of the summary can be read without starting an hour of measurement.

5.7 CLI-01: a mistyped flag terminates the driver, and a zero repetition count crashes it

Symptom. Three command lines against the released driver: one ended in `terminate` called, one divided by zero, and one produced a message that did not name the flag it was about.

Root cause. Three faults sharing a function. The numeric conversion threw a standard library exception from outside any handler, nothing rejected a repetition count below one, and the message was built without the flag's name.

Fix. Conversions go through a character based parse that refuses trailing text as well as text that is not a number at all, and name the flag in the exception. A repetition count below one is refused with a message that says what it would have done. The parse happens inside its own handler, separate from the one around the run, because it has to finish before the distributed runtime is initialised: the flags are what decide whether this process is part of a distributed job.

Verification. A test that runs the driver binary and asserts a non zero exit that is a status and not a signal, that `terminate` called appears nowhere, and that the message names the flag. It also runs the version flag, so that three refusals cannot be three failures to launch.

5.8 CLI-02: two ways out of the parser that the caller could not catch

Symptom. Found while writing a fuzz target that asserts the parser answers any command line with either an options structure or an exception. Two paths answer with neither: they print and exit. A fuzz harness cannot run past an exit, so those paths could not be exercised in process at all.

Root cause. The unfinished half of CLI-01. That finding put the parse inside a handler and made the numeric conversions throw; two older paths, a flag with no value and an unknown flag name, were left as they were. One function, two error protocols, and the second one bypassed the handler the first exists for.

Fix. Both paths throw the same argument error as the rest.

Verification. Twenty thousand generated command lines from a fixed seed, now including flags with no value after them, produce an options structure or a library error every time and nothing else. The test that runs the driver binary and checks what a user sees is unchanged and still passes.

5.9 TEST-01: the test framework had three defects, and nothing tested the test framework

Symptom. No failure, which is the shape of every defect in a checking mechanism: the suite was green and had been green through every phase. Three holes, found by reading the framework and the two copies of its helpers the suites had made.

Root cause. The first is a missing catch all, so a case that threw something outside the expected hierarchy took the process down instead of failing. The second and third are one root cause wearing two faces: the vector comparison helper was written twice, once per suite, and neither copy treated "these are not the same shape" as a difference. Both decided what to do about a length mismatch in the middle of a loop that was about element values, and both chose the answer that let the loop finish.

Fix. One helper in the framework, used by both suites, which reports a length difference as a difference. A catch all around every case. The framework's own test registers first of all, because a framework defect devalues every gate above it, and its deliberately failing cases print a banner around themselves so that failure lines inside a passing run read as deliberate.

Chapter 6

Provenance and measurement validity

This chapter is the audit that produced release 1.1.0. Not one of these entries is a crash. They are the reasons a number in the released report could not be reproduced, or was measuring something other than what it was labelled, or was smaller than the noise it was quoted against.

6.1 PROV-01: every published result row is stamped dirty

Symptom. The published summary held 850 rows and every one carried a `.dirty` stamp in its commit column, so the exact source that produced every published number did not exist as a commit in this repository. A clean build reproduced the stamp deterministically, so it was a standing condition of the tree rather than an accident of one session.

Root cause. Three independent mechanisms, all of which had to go together. The two report PDFs were tracked and at the same time matched by their own ignore lines, so a clean deleted two tracked files and every build after a clean was dirty before it compiled anything. The agent instruction files were untracked and unignored, and an untracked file dirties the tree exactly as a deleted one does. And the dirtiness check asked git about the whole tree, so a regenerated figure or a fresh summary counted towards the dirtiness of the binary, which makes the full build non idempotent on its own terms: the first run regenerates the tracked assets and the second run's sweep refuses to start.

Options. Filtering the status output inside the build system was rejected as a second copy of the ignore rules, written in another language, in a file nobody opens when they edit the first. Tracking the generated assets and the measurements as sources was rejected because it makes every report build produce a commit and hides exactly the churn the rule exists to detect.

Fix. The two PDFs are untracked and the published copies under the assets directory stay tracked, which is what the landing page links. The instruction files join the private working material block of the ignore file. The dirtiness check is given a pathspec, so only the sources that determine the binary's behaviour count.

Verification. Each of the six paths that used to dirty the tree is matched by a rule now, and the ignore checker names the line that catches each one.

6.2 PROV-02: an excluded directory that no later negation can re include

Symptom. Found by inspection while writing the rules for PROV-01, before the archiving phase could hit it. The ignore file excluded the results directory and re included three named files below it. The archiving step moves an existing 425 row measurement set into a subdirectory of it, and staging that with a whole tree add would have committed the new manifest and silently dropped all 425 archived rows, without the status output mentioning them.

Root cause. The exclusion matches the subdirectory itself, and git does not descend into an excluded directory, so a negation of a path inside it is never consulted: the walk stopped one level above. The rule reads as though it should work, which is what makes it worth an entry.

Fix. The directory is re included before any path inside it, and the manifest pattern is re included by name.

Verification. Probe files created under the results directory are listed by the untracked file listing under the new rules and are not under the old ones, and the ignore checker names the negation line that re includes each.

6.3 MEAS-01: Jacobi copies the whole state on the calling thread, every iteration

Symptom. Two readings of one defect. The published scaling block has Jacobi on OpenMP peaking below twice the serial time and then falling away, with no worker count moving it further. And the device comparison section puts host Jacobi beside device Jacobi under a normalised efficiency comparison, in the one section written to be hard to misquote, while the two were not running the same algorithm.

Root cause. The sweep function returned nothing, so a sweep had no way to tell the driver where it had put the new iterate. The only remaining way to say it was to put the iterate back where the driver already expected it, which the Jacobi solver did with a serial copy of the whole state, on the calling thread, once per iteration. At the sizes the device comparison uses that copy is hundreds of megabytes per iteration and it does not parallelise, so it is an Amdahl term the report attributed to the memory system.

Fix. The sweep function returns a view. The driver holds the current iterate and a spare, and after each sweep takes the current from the returned view, moving the old one into the spare when the two differ. Jacobi sweeps into the work buffer and returns it; every other sweep returns its first argument. Three places in the driver had to move with it, and only the first of them is obvious.

Verification. Two things had to be shown: that no iterate moved, which the equivalence suite asserts unmodified, and how much the copy was costing, which is recorded as a before and after in the phase record.

6.4 MEAS-02: Richardson evaluates the residual twice per iteration

Symptom. Richardson is timed against the other methods on the same problem at the same size, and its seconds per iteration are higher than the method’s arithmetic accounts for. The same run more than doubles in cost when the convergence check interval changes, and a method whose iterate does not depend on the check interval should not.

Root cause. Two operator applications per iteration where the method needs one. The solver computed the residual inside its sweep, because the step direction is the residual, and the shared driver computed it again at the end of the same iteration whenever the check fired, which at the default interval is every iteration. The second evaluation is at a different iterate, so it cannot simply be dropped without changing what the row reports.

Fix. Richardson runs its own loop: it updates from the residual it already holds, then evaluates the residual once at the new iterate, uses that norm for the convergence test when the check is due, and keeps the vector for the next update. One evaluation per iteration, reported at the iterate every other method reports at.

Verification. The iterates do not move: the arithmetic is the same operation over the same vector in the same order, and the evaluation has only moved from the top of one iteration to the bottom of the previous one.

6.5 MEAS-03: the symmetric methods were charged one sweep, and the mitigation the code claimed did not exist

Symptom. The driver computed updates as unknowns times iterations, and both the throughput and the achieved bandwidth are derived from it. Symmetric Gauss Seidel and SSOR perform two full sweeps per iteration, so both numbers were half of what those methods achieved. A comment in the solver header said, of exactly this, that the result rows record sweeps so the report can compare on equal work. There was no such column and the evaluation count was set to the iteration count, so the mitigation the comment described was not in the code.

Root cause. One number was being asked to mean two things and was given the value of neither. A row needs updates per unknown per iteration, which is what a work comparison divides by, and separately needs streams over the array per iteration, which is what a traffic model divides by. For nine of the twelve methods the two agree and the confusion is invisible. For the red black methods they differ: each colour pass writes half the unknowns and the pair writes each unknown exactly once, in two strided traversals of the whole array. One sweep of work, two passes over memory.

Fix. Two columns, sweeps and passes, documented at the field so a reader of the data can follow the definitions without reading a solver. The declaration is pure virtual, so a new solver cannot be added without stating both, and the device path reads the same declaration through the host solver of the same name rather than keeping a second table.

Verification. The two symmetric methods report two sweeps, the four red black and symmetric rows report two passes, conjugate gradient reports one more evaluation than iterations, and the two red black methods report one sweep, so their updates are unchanged. A unit case holds the whole table and fails if the registry gains a solver that does not declare both.

6.6 MEAS-04: a relaxation factor recorded on rows that never used one

Symptom. Every result row carried a relaxation factor. A Jacobi row carried Young's optimum for SOR at that grid, which Jacobi has no use for. A conjugate gradient row carried it too.

Root cause. The driver asked the SOR class for a factor for every row regardless of which solver ran. Eight of the twelve methods take none, so eight twelfths of the rows recorded a parameter the run never read, and two of the four that do take one recorded the wrong value, because they have their own defaults and were reported with the SOR optimum. The two that were right were right by coincidence of asking the class that happens to own their default.

Fix. The factor is a virtual on the solver, defaulting to zero, which is what the eight inherit. The driver prints the value when it is positive and leaves the field empty otherwise. The device path asks the host solver of the same name, and gets an empty field for exactly the kernels whose device code does not use one.

Verification. At a small grid, every one of the twelve is checked for the presence or absence of a factor and for its value where it has one, by a unit case rather than by reading the output.

6.7 MEAS-06: the byte model is undercounted, and the triad it is divided by pays the same charge it does not count

Symptom. The stencil declares three doubles per unknown per pass, and every achieved bandwidth in the repository is that figure times a work unit over a time. The same repository divides those bandwidths by a host triad that declares the same three doubles per element. Both are counted the same way, and if that way is wrong they are wrong together and in the same direction, which is why the error had never shown up as an inconsistency.

Root cause. Neither count charges read for ownership. A store to a line the cache does not hold has to fetch that line before it can modify it, so an array a pass writes without having read it first costs a read as well as a write. A Jacobi pass has exactly one such array, the output. The plain triad is in the same position for the same reason, and whether it really pays the charge is not asserted anywhere: it is the thing to be measured.

Fix. A second byte count beside the first, on the problem interface, with each one's derivation written out array by array. Every result row carries both, host and device. The existing column keeps dividing by the conservative count it has always divided by, and the report derives a second bandwidth from the second column, labelled counted beside declared. Neither replaces the other, so no published number changes underneath a reader.

6.8 MEAS-07: the probe's run to run spread is wider than the band it selects on

Symptom. Found immediately after building the instrument for MEAS-06, by running it three times on the same quiet machine. The selection statistic read 1.0990, then 1.0121, then 1.0601. The pre registered undecided band is 1.10 to 1.20 and is 0.10 wide. The spread of the statistic over three runs is 0.087, which is that band over again.

Root cause. Two contributions. The statistic as registered is a ratio of two figures each taken as the best over a worker sweep, and the two bests need not come from the same worker count: on one of those runs the two arms peaked at different counts, so the ratio compared two things differing in the store instruction and in the worker count at once. And a single best over one run carries the whole run to run variation of the machine.

Fix. None to the rule, deliberately: the thresholds and the outcome sentences were fixed before any measurement and are not revisited. What this entry required is that a decision be made and written down, before the publication measurement, on how the statistic is taken. That decision is the amendment of the following day: five repetitions of each probe at every worker count, alternating the two so a repetition's arms see the same machine; the statistic is the median of the per repetition ratios at the worker count where the plain probe's median is highest; and if the interval of those five ratios contains either threshold the outcome is unresolved whatever the statistic is. That last clause is ground rule 7 applied to a denominator.

Verification. The instrument itself is sound: the disassembly shows the streaming store and the fence. The non temporal arm is ahead at twenty two of the twenty four worker points across the three runs and behind at two, both in one run. The direction is the one read for ownership predicts and the exceptions sit inside the spread. It is the size of the effect, not its sign, that this instrument could not yet resolve.

6.9 MEAS-08: two pinning policies bound nothing and said they had

Symptom. The processor chooser returned minus one when it could not tell the performance cores from the efficiency ones, and every caller read that minus one exactly as it read the answer for "no pinning requested": skip the binding, count no failure, carry on. On this guest the classification never succeeds, so two of the policies bound no thread at all and printed a row whose pinning column named the policy.

Root cause. One sentinel for two answers that call for opposite responses. "Nobody asked for pinning" and "the classification this policy needs did not succeed" were both minus one, so no caller could tell them apart, and the only behaviour that serves the first is silence. A failure counter existed for a third answer, "the operating system refused", but nothing read it and no column carried it, so even the case that was detected was invisible.

Fix. The chooser returns an outcome and a processor number, and the outcome separates not requested, bound, not applicable and refused. One function turns a target into an outcome by attempting the binding, so the four backends no longer carry four copies of the rule. The

backend interface gains a pinning status the result row records. The two policies that do bind return exactly the processors they returned before, which is what keeps the published rows comparable with anything measured later.

Verification. A policy that needs a classification this machine does not yield now exits with a message saying so and writes no row. The three policies that bind write rows whose status column reads what they achieved.

6.10 MEAS-10: the timed region allocated on every call, and the state was only the largest part of it

Symptom. The driver timed the solve, and the solve allocated every full size vector it needed on entry and freed them on return. At the largest measured grid that is hundreds of megabytes mapped and unmapped inside the clock on every repetition. The untimed warm up could not help: the memory it faulted in was freed before the first timed repetition asked for its own.

Root cause. Measured rather than reasoned about, with a counting allocator armed at the two ends of the timed region. At twelve iterations Jacobi allocated forty five times, and the three state vectors are three of those forty five. The rest decompose exactly: three preconditions in the driver whose message literals are longer than the small string buffer, one reduction for the right hand side norm, two dispatches for the initial residual, and three dispatches per iteration. The state is most of the bytes and almost none of the count, and the count is what runs per iteration. The phase's own plan anticipated one of the six causes.

Fix. The solve borrows its workspace from the caller, allocated once outside the timed region, and the per dispatch and per precondition allocations are removed at their sources.

Verification. A gate that replaces the global allocation function and requires zero allocations between the two ends of the timed region, for all twelve solvers, on two backends. The same file carries a third case that allocates on purpose inside the armed window and requires the count to move, because a counter that counts nothing passes every test there is.

6.11 MEAS-11: dispersion was collected on every row and read by nothing

Symptom. The minimum and maximum time over the repetitions are written into every one of the 850 published rows and were read by no code in the repository: not the asset generator, not a chapter, not a document. The harness had been measuring how far each configuration moves between repetitions since the sweep was written, and throwing the answer away at the point where it would have qualified a number. Three headline claims of the report are smaller than the spread of the rows they were computed from.

Root cause. Two ground rules were written down and never expressed in code. Nothing in the pipeline could have caught it, because a report built from a generator that quotes only medians is indistinguishable from a correct one at every gate the project had: the tables are generated, the figures are generated, the numbers match the data, and the data is right. The

missing thing was a rule about what may be said, and a rule that lives only in prose is enforced by whoever last read the prose.

Fix. One definition of spread, in one helper, used by every table, every figure and both rules. Every timing table gained a spread column and every timing figure minimum to maximum whiskers. Bounds on a derived quantity are taken at the corners of its inputs' intervals, which is stated in a constant, in a comment and on the face of every figure that draws one. A separability helper returns either the signed effect or the phrase "not separable at this precision", deciding on the magnitude against the larger of the two rows' spreads, and every table that states a difference goes through it. Beside each such table the generator writes a verdict fragment, one sentence per comparison, so the prose can quote the generator instead of a number somebody typed while reading the table. The knee, which is the project's most repeated finding, is refit on a bootstrap of the repetitions behind each point and reported with an interval.

Verification. The dispersion table carries the count, the median and the maximum per block of the matrix, and deliberately no ninetieth percentile: the smallest block has six rows, and a ninetieth percentile over six is the single worst row wearing the name of a statistic. The generator has its own test, which asserts the spread helper on known values, the separability helper on a pair that separates and a pair that does not, that an effect with no measured spread never prints as a number, and that the bootstrap finds a planted knee and returns an interval containing it under both resamplings.

6.12 PROV-03: two generations in one summary, told apart by the order they were appended in

Symptom. The published summary held 850 rows across two commits, 425 each, and the asset generator chose between them by taking the commit of the last row in the file.

Root cause. Row order was standing in for a timestamp, and a timestamp was available and unread. The accumulation itself is deliberate and correct, since the commit is part of the resume identity, so a rebuild appends a generation rather than overwriting one and the history is kept. What was missing is that the file is a record of every generation while the report is a statement about one, and nothing made that difference explicit. Choosing silently is the worst of the three available behaviours, because it produces a report that is wrong in a way that leaves no trace.

Fix. The generator refuses when the summary holds more than one commit, names each with its row count, and names the path the superseded rows go to. With one generation present it sorts by the recorded time, so any later reading of "latest" reads a clock rather than an append order. The superseded rows moved to an archive beside the summary.

6.13 PROV-04: the committed manifest described a different session

Symptom. Four artifacts in this repository stated the host triad bandwidth for one machine and no two agreed. This entry is the mechanism behind two of the four.

Root cause. One manifest file with a fixed name, overwritten by whatever session ran last, and a bandwidth refresh step that updated the figures without recording that it had run. The committed manifest therefore described a later eight configuration re run rather than the 440 configuration sweep the published rows came from, and nothing in the file said which.

Fix. Sessions write a manifest named for their commit and their timestamp, and the refresh updates the manifest of the commit it refreshes rather than creating a second one for the same commit. The counts are written from one dictionary in one statement so they cannot drift, and the refresh records when it ran. The generator selects the manifest whose name carries the commit it is publishing and refuses when there is none, because every efficiency figure divides by a bandwidth from that session and there is no such number without it.

6.14 PROV-05: the manifest named a compiler that built nothing

Symptom. The interim session manifest records a GCC 16 trunk snapshot as the compiler of a binary that GCC 15 built, and records a rule of hyphens as the MPI version. Both would have travelled: the publication session writes the same fields, they are copied into the published assets, and the environment table of a report is exactly what a stranger uses to decide whether a number is reproducible on their machine.

Root cause. Two probes that answer a question they were never asked to verify. The harness held a literal list of commands and the first named a compiler, which is a second answer to a question the build system has already answered in its cache; it drifted the day the publication compiler changed and nothing could notice, because the field is a string nobody compares against anything. And the launcher on this image prints its help banner instead of a version, because the help file it wants is absent from the packages that should carry it, so the probe took the banner's first line, which is a rule of hyphens.

Options. Changing the literal to the current compiler was rejected: right today and silently wrong on the day the publication compiler moves, which is a change the plan already anticipates. Baking the compiler into the binary at configure time was rejected as more machinery than the question needs, since the build tree is already an input to the driver and its cache is the authority.

Fix. The compiler is read from the build tree's cache and whatever it names is run for its version, with the build tree taken as the parent of the binary the driver was pointed at. When the cache line is absent the field reads `unavailable` and a second field says why; it never falls back to a compiler name. For MPI, a first line carrying no digit is not a version whatever else it is, so the probe falls through to the information tool and the manifest records which probe answered. Both explanatory fields sit beside the values they explain, because a manifest is read by someone who does not have this repository open.

Verification. The probe run against the build tree this phase's gate builds and tests, with no sweep, names the released compiler the Makefile actually uses and the launcher's version through the fallback, and says which probe supplied each.

Chapter 7

Build, install and continuous integration

7.1 BUILD-02: the one flag target carried the machine's architecture into every consumer's build

Symptom. Not a failure, because until the install work nothing could consume the library: the released version shipped no install rule, no export and no package configuration file, so the defect had no way to reach anyone. It surfaced the moment the export was attempted.

Root cause. Two unrelated kinds of flag were in one interface target because until there was an install rule there was no reason to tell them apart. One kind is the numerical contract, which a consumer must compile with for the bit identity claim to be true of their own build, and since the library is about three quarters headers those have to travel with the headers. The other kind is this developer's build policy: the native architecture flag is a property of the machine the benchmark runs on, and warnings as errors is a promise about a warning set that a newer compiler will break in code the consumer did not write.

Fix. The contract target keeps the contraction flag and the language standard and nothing else. A separate developer target carries the warning set, the optimisation and architecture flags and the warnings as errors policy, is populated only when this project is the top level one, is linked privately, and is never installed.

Verification. The trap in the other direction is the expensive one, so it is checked rather than assumed: the compile line of the measured binary, of the backend factory and of the equivalence suite still carries all three of the flags that decide a number, so nothing measured moved.

7.2 BUILD-03: the export set refuses a target that links a static library it does not contain

Symptom. The first export naming only the two targets the plan called for stopped the configure dead, at generate time rather than at install time.

Root cause. The core library links the device library publicly, and the device library is a static one. A static library does not link its own dependencies, so whoever links the core archive must also link the device archive, and the build system will not write an imported target whose link interface names an archive it has not been told how to find. The rule is not about the device toolkit; it is about static libraries, and it fires on any public link to one that stays outside the set.

Fix. The install target list is built up rather than written out, with the device library appended when it exists, so the export set matches what the build produced.

Verification. It bit, and it was made to bite again on purpose: with the guard disabled a configure fails with exactly the original message, and with it restored the staged install writes both archives and the targets file declares both. A throwaway consumer declaring only C++ links against the staged install and runs.

7.3 BUILD-04: a private dependency of a static library is not private to the export set

Symptom. With the device library in the export set, the same command failed again, naming the target that had just been created to be kept out of it.

Root cause. A private link on a static library does not mean what it looks like it means. A static library performs no link of its own, so the build system has to tell whoever links the archive later about every dependency, private ones included; it records them in the interface link list wrapped in a link only generator expression, which suppresses the compile side of the usage requirement and keeps the link side. The export step then validates every target named there. The target that exists precisely so the architecture flag never crosses the install boundary was blocking the install.

Fix. The private link is wrapped in a build interface expression. The build tree is unchanged and the exported interface records an empty link only entry.

Verification. The three compile lines that decide a measured number are unchanged, and the exported package files contain neither the architecture flag nor the warnings as errors policy.

7.4 BUILD-05: the self check for the allocation counter was deleted by the compiler it was meant to check

Symptom. The first run of the suite under a third compiler failed the case that exists to prove the allocation counter counts: it reported that nothing had been allocated inside a window that allocates on purpose.

Root cause. The compiler deleted the allocation. A new expression is one of the few constructs an implementation may remove outright, when the storage does not escape. Here the array was written once, read never and deleted a few lines later, so the optimiser paired the allocation with the deallocation and dropped both. Nothing reached the replacement operator, the counter

stayed at zero, and the check correctly reported that it had no evidence. Neither of the other two compilers performs the elision on this shape, which is why one compiler had been enough to keep it hidden.

Fix. A file scope volatile pointer assigned immediately after the new expression and cleared after the delete. Four lines, and nothing about what the gate asserts changes.

7.5 BUILD-06: a unit test named a backend the build is allowed not to contain

Symptom. In the same run, a case failed because it asked the factory for the OpenMP backend by name in a build that has no OpenMP runtime.

Root cause. The build system is written to configure without that backend when the runtime is absent, and prints that it is doing so. The test then asked for it unconditionally, so a configuration the build supports on purpose became a test failure.

Fix. The case is compiled only when the backend is present. When it is not, a case of the same shape takes its place, named so the skip is visible in the output, and it requires that the backend really is absent from the list the library reports. A build that has the backend can therefore never take the skip path silently.

7.6 BUILD-07: a default argument on a virtual function, and the override that omitted it

Symptom. The same call, on the same object, through the two static types it has: one compiles and one does not.

Root cause. The base declaration carried defaults for its last three parameters and the override declared none. A default argument is part of the declaration the call is resolved against, which is chosen from the static type of the expression, so two declarations of one function disagreed about how many arguments it takes. Here that shows as a compile error. The worse form of the same fault is silent: had the override supplied different defaults, the call would have compiled through both types and done different things.

Fix. The base function is a non virtual wrapper carrying the defaults, and the virtual behind it is protected and has none. The four call sites are unchanged, since they call the wrapper by the same name as before.

Verification. A case that records what the implementation was handed and requires the two spellings of the call to agree, argument by argument, and to be the documented defaults.

7.7 CI-01: the CUDA job reported success having compiled zero CUDA

Symptom. The job was green on every run of the released version and had never compiled a line of device code on the runner. The entire device half of the report was untested by automation and the badge said otherwise.

Root cause. Three lines of the workflow, and the shape of them rather than any one of them: the toolkit install was allowed to fail without failing the step, so every subsequent step found no compiler and skipped itself, and the job ended green having done nothing.

Fix. The install step is unconditional and swallows nothing. The host compiler and the C++ compiler of that build are pinned to the newest the runner's toolkit accepts. If the toolkit ever leaves the runner's archive the job goes red, and that is the correct outcome: it is what the job is for.

Verification. The workflow cannot be run from this machine, so it is verified for syntax and every command in it is verified locally through the same targets. The failure path is verified by construction: an install that fails is a step that fails, and no branch is left that can swallow it.

7.8 CI-02: the reports job never compared the reports it rebuilt against the ones it ships

Symptom. The job regenerated every asset from the committed data, checked that the output files existed and were not empty, and stopped. Four artifacts stated the host bandwidth, no two agreed, and continuous integration was green throughout.

Root cause. The job's own comment explains, correctly, why the charts are not compared byte for byte: a rendering depends on the library version and the fonts installed, so bytes produced on a runner will never equal bytes produced on a development machine. That argument is right about images and was allowed to stand in for a comparison of the numbers, which are what the documents are for.

Fix. A script that extracts the text of both documents, drops a short and explicitly listed set of volatile lines, takes every numeric token in reading order and fails on the first divergence, printing the page and the line from both sides. Tokens are compared as an ordered sequence, so a value that moved between two tables is a divergence and not a match. A comparison that could not be made exits with a different status from a comparison that failed, so a missing extraction tool can never read as agreement.

7.9 CI-03: the compiler, the runner image and every action were whatever the day supplied

Symptom. Three unpinned things in one workflow, none of which produced a failure and all of which made a green run mean less than it looked: a loop that chose the newest compiler the runner happened to have, a floating runner image, and action references by mutable tag.

Root cause. The compiler chooser was written when the project targeted a trunk snapshot and the runner had nothing near it, so the loop was a way to build with the best available. What it encodes is that the compiler under test is a property of the runner image and changes when the image does, silently and between two runs of the same commit.

Fix. A pinned image, a matrix of three named compilers by two build types where each leg asserts the compiler's version against the one it claims, and every action pinned by commit with the tag as a trailing comment, with an automated update policy so the pins move deliberately.

Chapter 8

The documents themselves

8.1 DOC-01: a hand typed table under the sentence saying nothing was typed by hand

Symptom. The results chapter opened by saying that every table and figure in it is generated and that nothing there is typed by hand, thirty lines above a table of the host triad against worker count that was typed by hand. The landing page carried the same table again and the methodology document carried a third copy of one of its figures. Four artifacts stated the host bandwidth and no two agreed, and the value the chapter's central narrative rested on appears in no machine generated artifact anywhere in the repository. The generated table shipped inside the published document peaks at a worker count the phase record describes as withdrawn because it failed to reproduce, so the report showed a retracted measurement as evidence for the claim that replaced it.

Root cause. Two mechanisms, and the second is the one that matters. There was no generated table of the triad against worker count at all: the generator emitted the peak of each probe and one detail string, and the per worker figures were only ever inside that string, so a sentence that wanted the shape of the curve had nowhere to get it and was typed. And nothing could notice, because every gate the repository had was a presence check.

Fix. The generator emits the scaling table from the same manifest the peak comes from, with the spread of each probe's repetitions beside each worker count, and the chapter inputs it where the typed table stood. Beside it the generator writes one command per measured figure the prose quotes, and the document inputs that in its preamble, so a sentence asks for a command instead of carrying a number. The methodology document's tables move into a generated region rewritten by the same script.

Verification. The property that matters is not that the numbers are now right. It is that a figure the generator cannot derive is a build failure rather than a stale sentence, and that the peak in the scaling table and the peak in the bandwidth table are one expression over one manifest and cannot disagree.

8.2 DOC-02: no prior art and no baseline

Symptom. Not one library this work could be compared against was named anywhere in the repository. The bibliography held sixteen entries: classic textbooks, two standards, the rooftop paper and the STREAM paper. All sound, and not one of them a library this work could be measured against.

Root cause. Two separate gaps that presented as one absence. There was no prior art for the central claim, although deterministic fixed chunk reduction is established technique and the reproducible summation literature, a vendor library’s reproducibility mode and two portability layers all predate this. And there was no performance baseline, so a statement that a solver reaches some percentage of the host’s streaming bandwidth cannot be read as good or bad.

Options. Citing the prior art and leaving the baseline was rejected as a complete answer: it fixes the second gap not at all and the first without admitting the second. Building the baseline now was rejected for this release on the recorded split, since it belongs to the release that also brings the container and the assembly work.

Fix. A related work section in the report and a longer document beside it, citing the vendor reproducibility mode, the reproducible summation papers and the binning algorithm behind ReproBLAS, the two portability layers and the three solver libraries, with metadata checked against the publisher or the project’s own documentation in every case. It then states what this project adds, which is not the deterministic reduction: one numerical invariant held across seven execution models including a graphics device and a distributed backend, asserted as exact equality in a test suite rather than described, and priced with its run to run spread. It says plainly that no baseline is in this report and names the release that brings one.

Chapter 9

What the faults have in common

Five of the entries from the original build produced plausible numbers rather than obvious failures: the eigenvector right hand side, the relaxation default, the resume logic, the tolerance below the arithmetic floor, and the fused multiply add discrepancy. Those are the ones worth generalising from.

Assert the shape, not just the outcome. The eigenvector fault was caught because a test asserted the iteration count should *grow* with the grid, not merely that the solver converged. A test checking only convergence would have passed forever while measuring nothing.

Exact comparisons find what tolerances hide. The one bit fused multiply add difference would have vanished into any reasonable tolerance. It was visible only because the equivalence suite asserts bit identity, and following it led to a real weakness in the project's central claim.

Measure the floor before choosing the target. Asking for accuracy below what the arithmetic can deliver produces a failure that looks like a regression in whatever component changed most recently.

Defaults should defer, not decide. A shared options structure whose defaults suit whichever component was written first will silently mis-configure the others.

Time the fast path, do not merely observe it. The resume logic reported correct counts while doing no work correctly. It survived until someone timed a restart instead of reading its output.

9.1 What the repair added to that list

The entries from the audit of the released tree have a different shape, and they generalise differently. Almost none of them was a failure, and several had been green through an entire release.

A gate that only checks for presence checks nothing. The reports job regenerated every asset, confirmed the files were not empty, and compared them against nothing. The asset generator wrote tables from the data and could not see a table typed into the chapter beside

them. Four artifacts disagreed about one measurement for the life of a release, under continuous integration that was green the whole time. A check that cannot fail for the reason you care about is not a check.

A rule that lives only in prose is enforced by whoever last read the prose. The minimum and maximum time were recorded on every row from the first sweep and read by nothing, while three headline claims sat inside the spread they would have qualified. The remedy is not more discipline; it is one definition of spread in one helper that every table and every figure calls, and a phrase the generator prints when the rule fires.

A number that cannot be regenerated is a number that will drift. Every typed figure in this project drifted from its source, in the same direction, without anyone noticing. The repair is not to correct them but to remove the mechanism: a sentence asks the generator for a command, and a figure the generator cannot derive is a build failure.

One sentinel for two answers is a decision nobody can make. Minus one meant both "no pinning was requested" and "the classification this policy needs did not succeed", and the only behaviour that serves the first is silence, so the second was silent too and two policies bound nothing while their rows said they had.

Provenance is part of the measurement. Every published row carried a commit stamp saying its source was not in the repository's history, and the manifest beside it described a different session. Neither is visible in a number. Both make the number unreproducible, which is the property the number was published for.

Fix the convention, not the instance. Where a defect had already been copied, the repair went to the shared mechanism rather than to the copies: one comparison helper instead of two, one place that attempts a binding instead of four, one definition of spread instead of a rule in a document. Adding a second correct copy beside a first incorrect one is how a defect becomes a convention.